

universidad de león Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

Ingeniería del software seguro en la era de la IA generativa

Diseño y Programación Seguras

Correctitud y seguridad

Francisco Javier Rodríguez Lera ▪ Camino Fernández Llamas



Instituto de Investigación e Innovación en Ingeniería

Escuela de Ingenierías

universidad de León

Objetivos de aprendizaje

Al terminar el capítulo, el estudiante será capaz de:

- **Distinguir** las ocho nociones de «correcto» y sus implicaciones.
- **Formular** precondiciones, postcondiciones e invariantes como contrato.
- **Explicar** la lógica de Hoare y sus límites.
- **Demostrar** con el caso transversal que pasar las pruebas no es ser seguro.
- **Distinguir** requisitos explícitos de implícitos en un *prompt*.

«Correcto» es ambiguo

«El programa funciona» puede significar que compila, que da el resultado esperado en los casos probados, que resiste entradas malformadas o que soporta a un adversario.

- Son propiedades **distintas**; confundirlas origina fallos de seguridad.
- La clasificación en ocho nociones es una **Propuesta docente**: una taxonomía operativa, no un estándar cerrado.

Ocho nociones de corrección

Noción	Pregunta	Qué deja fuera
Sintaxis	¿Programa bien formado?	Todo el significado
Compilación	¿Respeta reglas estáticas?	Corrección funcional; entradas adversarias
Funcionalidad	¿Salida esperada en casos probados?	Casos no probados; seguridad
Correctitud	¿Satisface la especificación?	Que la especificación sea la adecuada
Robustez	¿Comportamiento controlado ante entradas inválidas?	Adversario dirigido; confidencialidad
Seguridad	¿Resiste a un adversario?	Lo no cubierto por el modelo de amenaza
Portabilidad	¿Igual en otro entorno?	Seguridad en el entorno original
Mantenibilidad	¿Comprensible y modificable?	Corrección actual

No forman una escalera

- **Compilar no es acertar**: no garantiza corrección ni ausencia de comportamiento indefinido.
- **Pasar las pruebas no es ser correcto ni seguro**: las pruebas juzgan un conjunto *finito* de casos; la seguridad exige resistir a un adversario.
- **La robustez no es la seguridad**: degradar de forma controlada ante un dato inesperado $\neq$ mantener propiedades ante un atacante que *elige* ese dato y el entorno.

Sin un modelo de amenaza explícito, «seguro» carece de referente y la afirmación no es verificable (ASVS [2], SSDF [1] orientan, pero no sustituyen ese modelo).

¿Robusto implica seguro?

- Un programa que no se cae ante una entrada malformada, ¿es seguro?
- ¿Qué añade la palabra «adversario» a la de «entrada inesperada»?

Especificación: precondición, postcondición, invariante

Precondición P. Condición que debe cumplirse *antes* de ejecutar C; supuesto sobre el estado inicial o la entrada. Si no se cumple, C no está obligado a nada.

Postcondición Q. Condición que debe cumplirse *después*, si P se cumplió al empezar.

Invariante I. Propiedad que permanece verdadera en puntos definidos (p. ej., antes y después de cada iteración).

La pareja (P, Q) es un **contrato**: P obliga al llamante; Q obliga al implementador.

Especificación del caso transversal

Conversión de CSV a XLSX:

- **Precondición:** el fichero existe, es legible, tiene cabecera válida, codificación declarada y tamaño dentro del límite.
- **Postcondición:** existe un XLSX en el directorio de salida, confinado a él, con las filas y valores esperados, y *sin* entradas no confiables interpretadas como fórmulas.
- **Invariante del bucle:** toda fila escrita ha pasado por validación; las celdas no confiables se escriben como datos, no como fórmulas.

La postcondición ya incorpora un requisito de *seguridad* (confinamiento y neutralización de fórmulas). Una especificación que solo describe el «camino feliz» es insuficiente.

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

¿Quién establece la precondición?

- Si el llamante no cumple la precondición, ¿de quién es el fallo?
- ¿Qué diferencia hay entre una poscondición informal y una prueba formal?

Lógica de Hoare: el triple

$\{P\}\ C\ \{Q\}$: si el estado inicial satisface P y C *termina* normalmente, el estado final satisface Q.

- **Correctitud parcial**: no afirma que C termine.
- **Correctitud total**: añade la garantía de terminación.
- Para bucles se usa un invariante I que se conserva antes y después de cada iteración.
- La corrección es siempre *relativa a la especificación*.

Permite demostrar corrección sobre *todo* el dominio de P, no sobre una muestra: «para toda entrada que cumpla P, la salida cumple Q».

Límites de la lógica de Hoare

- **Relatividad a la especificación**: el teorema dice «C cumple Q», no «Q es lo que se necesita». Hereda los errores de la especificación.
- **Propiedades de una sola ejecución**: la seguridad *relacional* (p. ej. ausencia de filtración) no se expresa con un triple estándar.
- **Supone semántica bien definida**: en C, el comportamiento indefinido (desbordamiento con signo, accesos fuera de límites) invalida el razonamiento; las reglas CERT [5] tratan de eliminarlo.
- **Concurrencia**: requiere variantes (separación concurrente, *rely-guarantee*) con supuestos adicionales.
- **Coste y alcance**: la verificación completa es costosa; se combina con especificaciones parciales, aserciones y pruebas.

¿Demuestra Hoare que un programa es seguro?

- ¿Qué demuestra exactamente un triple $\{P\}\, C\, \{Q\}$ correcto?
- Señale dos propiedades de seguridad que el triple *no* captura.

Pasar las pruebas no es ser seguro

Una función que convierte CSV en XLSX puede *superar* sus pruebas funcionales y seguir siendo insegura. Las pruebas comprueban lo esencial desde el punto de vista funcional: el fichero se crea, las columnas son correctas, las filas coinciden.

Pruebas verdes $\neq$ seguro. Superar una batería finita demuestra solo que *esos* casos pasan. Las pruebas cubren el camino feliz; la seguridad debe resistir el caso que el adversario elige.

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

Versión vulnerable que supera las pruebas

```python
import csv, os
from openpyxl import Workbook

def convertir_csv_a_xlsx(ruta_entrada, dir_salida, nombre_salida):
    # V1. Sin limite de tamano: carga el fichero entero en memoria.
    with open(ruta_entrada, newline="", encoding="utf-8") as f:
        filas = list(csv.reader(f))
    libro = Workbook(); hoja = libro.active
    for fila in filas:
        hoja.append(fila)          # V2. Formula injection (=...)
    ruta_salida = os.path.join(dir_salida, nombre_salida)  # V3. Path
        traversal
    libro.save(ruta_salida)        # V4. sigue enlaces simbolicos
    return ruta_salida
```

Listing 1: Conversión CSV a XLSX insegura (material didáctico VULNERABLE; solo en entorno aislado).

Las cuatro debilidades que las pruebas no ven

- **V1 - agotamiento de recursos**: cargar el CSV entero agota la memoria; las pruebas funcionales no miden consumo.
- **V2 - inyección de fórmulas**: `openpyxl` puede guardar como fórmula una cadena que empieza por = (CWE-1236); las pruebas usan datos benignos y no inspeccionan el tipo de celda.
- **V3 - *path traversal***: un `nombre_salida` con `../` escapa del directorio; las pruebas no lo ejercitan.
- **V4 - enlaces simbólicos / posible TOCTOU**: el listado no valida la ruta; el TOCTOU aparecería en una variante que comprueba y luego usa sin preservar la identidad del objeto (CWE-367; enlaces CWE-59).

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

Un oráculo más preciso

- Una prueba negativa debe comprobar el *tipo* de la celda resultante, no solo que el archivo se creó.
- Una entrada con `=1+1` puede generar una celda con `data_type == "f"` al cargar el XLSX con `openpyxl`.
- Este comportamiento depende de la biblioteca y debe verificarse en la versión utilizada.

La creación del fichero, por sí sola, no constituye una prueba de seguridad.

¿Basta añadir pruebas de seguridad?

- Añadir entradas con fórmulas, rutas con `..`, ficheros desproporcionados y enlaces simbólicos, ¿convierte el programa en seguro?

Requisitos explícitos e implícitos

Explícitos. Los que se enuncian: «leer un CSV, normalizar y generar un XLSX»; «no confiar en el CSV, escapar fórmulas, limitar tamaño y tiempo».

Implícitos. Los que se dan por supuestos: que la salida no permita *path traversal*, que el proceso termine en tiempo y memoria acotados, que el código sea comprensible.

Su incumplimiento constituye un fallo aunque no aparezcan en el enunciado.

Cobertura de pruebas por tipo de requisito

Tipo	Ejemplo	¿Lo comprueban las pruebas?
Explícito funcional	«Leer el CSV y generar un XLSX»	Sí, con casos normales
Explícito de seguridad	«Escapar fórmulas, limitar tamaño»	Solo con pruebas específicas
Implícito de seguridad	«No permitir *path traversal*»	No, salvo que se haga explícito
Implícito de recursos	«Terminar en tiempo y memoria acotados»	No
Implícito de mantenimiento	«Ser comprensible por un tercero»	No

El prompt es una especificación incompleta

- Un *prompt* es una especificación en lenguaje natural; suele omitir requisitos implícitos.
- Su *completitud* debe evaluarse, no presumirse: «convierte este CSV en Excel» describe lo funcional y omite lo de seguridad.
- El modelo completa los huecos con las opciones más plausibles según sus datos de entrenamiento; la plausibilidad estadística no coincide necesariamente con la seguridad.

Hipótesis respaldada por la literatura sobre seguridad del código sugerido por asistentes [3, 4], sin ser un resultado medido aquí.

Tres consecuencias, con distinto estatus

- **(Hipótesis)** Un modelo que no recibe un requisito de seguridad tiende a no incorporarlo; debe contrastarse con un protocolo de evaluación.
- **(Inferencia metodológica)** Las pruebas derivadas solo de requisitos explícitos pueden dejar fuera propiedades no especificadas.
- **(Propuesta docente)** Quien usa un asistente debe escribir los requisitos de seguridad *antes* de pedir el código y revisar el resultado contra ellos.

La IA abarata la *producción*, pero no *adivina* la especificación. Cuanto más se delega la escritura, más se desplaza el trabajo crítico hacia la especificación y la validación.

Síntesis del capítulo

- «Correcto» designa al menos **ocho propiedades** que no implican unas a otras: compilar no es acertar; pasar pruebas no es ser seguro.
- Precondiciones, postcondiciones e invariantes convierten el comportamiento en una especificación contrastable; Hoare le da forma matemática, pero es *relativa a la especificación* y de alcance limitado.
- El caso CSV→XLSX puede pasar sus pruebas y seguir siendo inseguro (recursos, fórmulas, rutas, enlaces).
- **(Hipótesis)** Un *prompt* es una especificación incompleta; lo que no se enuncia tiende a no aparecer ni en el código ni en las pruebas.

Actividad

1. Elija tres pares de nociones y construya un contraejemplo que satisfaga la primera y no la segunda.
2. Redacte P, Q e I para la conversión CSV$\rightarrow$XLSX; indique qué requisitos de seguridad incluye y cuáles deja fuera.
3. Explique por qué un triple $\{P\}C\{Q\}$ correcto no implica seguridad; señale dos propiedades que no captura.
4. Enumere tres pruebas que detectarían V2, V3 y V4; distinga enlace simbólico de TOCTOU.

¿Qué evidencia justificaría confiar en el código generado?

- Si un asistente produce código que pasa las pruebas, ¿qué falta para aceptarlo?

Referencias I



National Institute of Standards and Technology.

Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities.

Technical Report SP 800-218, NIST, 2022.

Norma / marco de referencia.



OWASP Foundation.

Application security verification standard (asvs), 2025.

Guía de referencia (versión estable 5.0.0, 2025).



Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri.

Asleep at the keyboard? assessing the security of github copilot's code contributions.

In *IEEE Symposium on Security and Privacy (SP)*, 2022.

Estudio experimental; sus resultados no se extrapolan automáticamente a versiones posteriores.

Referencias II



Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh.

Do users write more insecure code with ai assistants?

In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2785–2799, 2023.

Estudio con participantes; no es una estimación universal del efecto.



SEI / CERT Division, Carnegie Mellon University.

Sei cert c coding standard.

Technical report, 2016.

Edición CERT C 2016.